TP Data Science / Bases de données

Acquisition, stockage et exploitation de données avec Google Colab et SQL

Mini-projet : les avoirs du FMI en monnaie chinoise (1984 - 2026)

Source des données	Fonds monétaire international (FMI) : fichier **extportal.xls**, « China: IMF's Holdings of Currency », situation au 30 septembre 2026, unité : DTS (droits de tirage spéciaux, SDR)
Chaîne de traitement	Source FMI → téléchargement → Google Drive → Google Colab → Pandas → SQLite → requêtes SQL → analyse
Outils	Python 3, Pandas, sqlite3 (SQLite 3.45), Matplotlib
Contenu	Parties 1 à 7 : choix du dataset, Drive, exploration, base SQLite, 15 exercices SQL, analyse (3 graphiques), livrables

Comment lire ce document. Tous les résultats chiffrés (tableaux, moyennes, classements) proviennent de l'exécution réelle du code sur le fichier fourni ; aucune valeur n'est estimée à la main. Le fichier de base de données produit (`tp_sql.db`) est livré avec ce PDF.

Limite à signaler : le montage de Google Drive (`drive.mount`) ne peut être exécuté que dans Colab. Le code correspondant est donné tel qu'il doit être utilisé dans Colab, mais les résultats ont été obtenus en exécutant le même traitement en local sur le même fichier.

Partie 1 - Recherche et téléchargement du dataset

1.1 Source et dataset retenu

Le dataset provient du **Fonds monétaire international** (portail de données : https://www.imf.org/en/Data). Il a été exporté sous le nom `extportal.xls` et décrit, pour la Chine, les **avoirs du FMI en monnaie chinoise**, répartis entre plusieurs comptes, à chaque fin d'année de 1984 à 2025, puis au 30 septembre 2026.

1.2 Description des variables

Colonne d'origine	Nom SQL	Type	Description (à confirmer dans la documentation du FMI)
Date	date_obs	Date	Date d'observation (31 décembre de chaque année, puis 30/09/2026)
Securities	securities	Entier	Avoirs sous forme de titres (valeur constante : 0)
Nbr 1 Account	nbr1_account	Entier	Avoirs de la monnaie sur le compte n° 1 du FMI (composante principale)
Nbr 2 Account	nbr2_account	Entier	Avoirs sur le compte n° 2 (montants très faibles)
CVA Account	cva_account	Entier	Compte d'ajustement de valorisation (positif ou négatif)
CVA Nbr 2 Account	cva_nbr2_account	Entier	Ajustement de valorisation lié au compte n° 2
Total Holdings	total_holdings	Entier	Total des avoirs ; somme des 5 colonnes précédentes (vérifié : écart nul pour les 43 lignes)

Deux variables ont été ajoutées lors de la préparation : `annee` et `decennie` (variable catégorielle : 1980s, 1990s, ...), ainsi que `signe_cva` (Positif / Négatif / Nul).

1.3 Questions de la partie 1

Question	Réponse
Quelle est la source du dataset ?	Le FMI (portail de données du FMI, fichier extportal.xls).
Quel est le domaine étudié ?	Finance internationale et économie monétaire : position du FMI en monnaie d'un pays membre, la Chine.
Combien d'observations et de variables ?	**43 observations** (1984-2026) et **7 variables** d'origine (10 après préparation).
Quelles analyses sont possibles ?	Évolution temporelle du total, comparaison par décennie, identification des années extrêmes, part de chaque compte, signe et poids de l'ajustement CVA, variations d'une année à l'autre.

1.4 Justification du choix et écarts aux critères

Le jeu de données est public, de source institutionnelle fiable, et son format (série annuelle de montants) se prête naturellement aux agrégations SQL (`AVG`, `MIN`, `MAX`, `GROUP BY`). Deux critères du sujet ne sont toutefois pas satisfaits strictement, et il faut le dire :

• **Moins de 100 observations** : 43 lignes seulement. Le sujet autorise cet écart « sauf justification » : une série annuelle sur 42 ans ne peut pas contenir plus de lignes ; les 43 lignes suffisent pour tous les exercices.
• **Pas de variable catégorielle native** : toutes les colonnes sont numériques ou temporelles. Les catégories (`decennie`, `signe_cva`) ont donc été construites à partir des données.
• La colonne `securities` vaut toujours 0 : elle n'apporte aucune information et n'est utilisée dans aucune analyse.

Partie 2 - Stockage sur Google Drive et chargement dans Colab

2.1 Étapes à réaliser

1. Sur https://drive.google.com, créer le dossier `TP_SQL_Dataset`.
2. Y déposer `extportal.xls`.
3. Ouvrir un nouveau notebook sur https://colab.research.google.com.
4. Monter Drive, vérifier l'existence du fichier, puis le charger avec Pandas.

2.2 Code de la cellule 1 : montage de Drive et vérification

```
from google.colab import drive
drive.mount('/content/drive')          # autoriser l'accès dans la fenêtre Google

from pathlib import Path
chemin = Path('/content/drive/MyDrive/TP_SQL_Dataset/extportal.xls')

if not chemin.is_file():
    raise FileNotFoundError(f'Fichier introuvable : {chemin}')
print('OK, fichier trouvé :', chemin, '-', chemin.stat().st_size, 'octets')
```

2.3 Code de la cellule 2 : chargement avec Pandas

Piège rencontré. Malgré son extension `.xls`, le fichier n'est **pas** un vrai classeur Excel : c'est un texte séparé par des tabulations, précédé de 4 lignes de titre. `pd.read_excel()` échoue avec l'erreur « Excel file format cannot be determined » (vérifié). Il faut utiliser `read_csv` avec `sep='\t'`, ignorer les 4 premières lignes et déclarer la virgule comme séparateur de milliers :

```
import pandas as pd

df = pd.read_csv(chemin, sep='\t', skiprows=4, thousands=',')
display(df.head(10))
print('Lignes   :', df.shape[0])
print('Colonnes :', df.shape[1])
```

Vérifications obtenues : Lignes : 43 ; Colonnes : 7. Les 6 colonnes numériques sont bien lues comme entiers (`int64`) grâce à `thousands=','`.

Partie 3 - Exploration et préparation des données

3.1 Les 10 premières lignes (`df.head(10)`)

Date	Securities	Nbr 1 Account	Nbr 2 Account	CVA Account	CVA Nbr 2 Account	Total Holdings
September 30, 2026	0	22 629 291 897	47 166	-175 346 492	-366	22 453 992 205
December 31, 2025	0	22 492 513 935	47 161	-172 565 449	-361	22 319 995 286
December 31, 2024	0	23 000 140 155	46 800	0	0	23 000 186 955
December 31, 2023	0	23 266 670 657	46 800	0	0	23 266 717 457
December 31, 2022	0	22 490 606 714	46 800	0	0	22 490 653 514
December 31, 2021	0	23 219 946 714	46 800	0	0	23 219 993 514
December 31, 2020	0	24 255 978 451	47 905	-568 031 737	-1 105	23 687 993 514
December 31, 2019	0	24 512 331 862	45 254	842 662 408	1 546	25 355 041 070
December 31, 2018	0	24 464 085 638	44 651	1 200 179 133	2 149	25 664 311 571
December 31, 2017	0	26 972 619 141	47 362	-314 016 146	-562	26 658 649 795

3.2 Colonnes, types, dimensions, valeurs manquantes, doublons

colonne	type	valeurs_manquantes
Date	object (texte)	0
Securities	int64	0
Nbr 1 Account	int64	0
Nbr 2 Account	int64	0
CVA Account	int64	0
CVA Nbr 2 Account	int64	0
Total Holdings	int64	0

Dimensions : **43 lignes x 7 colonnes**. Valeurs manquantes : **0**. Lignes dupliquées (`df.duplicated().sum()`) : **0**. Dates dupliquées : **0**.

```
df.dtypes
df.isna().sum()
df.duplicated().sum()
df.describe()
```

3.3 Statistiques descriptives (`df.describe()`)

index	moyenne	écart-type	min	médiane	max
Securities	0	0	0	0	0
Nbr 1 Account	9 330 038 921	8 942 461 524	803 784 756	5 810 831 517	26 972 619 141
Nbr 2 Account	31 333	19 906	1 544	44 841	50 582
CVA Account	66 122 808	398 774 910	-701 881 549	0	1 200 179 133
CVA Nbr 2 Account	237	2 164	-4 081	0	8 271
Total Holdings	9 396 193 300	8 949 345 621	861 883 648	5 396 222 985	26 658 649 795

Lecture. La moyenne du total est d'environ 9.40 Md de DTS pour un écart-type de 8.95 Md : la dispersion est quasi égale à la moyenne, ce qui indique une série fortement non stationnaire (ordres de grandeur très différents selon les époques). `Securities` est constante (écart-type nul). `Nbr 2 Account` ne dépasse jamais 50 582 DTS, soit un montant négligeable devant `Nbr 1 Account` (jusqu'à 26,97 Md). Enfin `Total Holdings` est exactement la somme des 5 composantes.

3.4 Transformations avant l'import dans SQLite

1. Renommer les colonnes en *snake_case* sans espaces (SQL n'aime pas « Nbr 1 Account »).
2. Convertir `Date` (texte anglais « December 31, 2024 ») en date, puis en texte ISO `AAAA-MM-JJ` : SQLite n'a pas de vrai type date et l'ordre alphabétique d'un texte ISO coïncide avec l'ordre chronologique.
3. Créer `annee`, `decennie` et `signe_cva`.
4. Trier par date croissante.

```
df.columns = ['date_obs','securities','nbr1_account','nbr2_account',
              'cva_account','cva_nbr2_account','total_holdings']

df['date_obs']  = pd.to_datetime(df['date_obs'], format='%B %d, %Y')
df['annee']     = df['date_obs'].dt.year
df['decennie']  = (df['annee'] // 10 * 10).astype(str) + 's'
df['signe_cva'] = df['cva_account'].apply(
    lambda v: 'Positif' if v > 0 else ('Negatif' if v < 0 else 'Nul'))
```

```
df['date_obs']  = df['date_obs'].dt.strftime('%Y-%m-%d')
df = df.sort_values('date_obs').reset_index(drop=True)
```

Vérification : répartition de `signe_cva` : 19 positifs, 17 négatifs, 7 nuls (total 43). Répartition par décennie : 1980s = 6, 1990s = 10, 2000s = 10, 2010s = 10, 2020s = 7 (2020 à 2025 plus le 30/09/2026).

Partie 4 - Création de la base de données SQLite

4.1 Rôle des outils

Outil	Rôle
`sqlite3`	Module standard de Python : crée/ouvre une base SQLite (un simple fichier .db, sans serveur) et exécute des ordres SQL via une connexion.
`pandas`	Manipule les données en mémoire sous forme de DataFrame ; lit les résultats SQL avec `read_sql_query()`.
`to_sql()`	Méthode de DataFrame qui écrit le DataFrame dans une table SQL. `if_exists` règle le comportement : *fail*, *replace* (supprime et recrée) ou *append* (ajoute les lignes). `index=False` évite d'écrire l'index Pandas.

4.2 Création de la base, de la table et import

Pour que la table ait un schéma maîtrisé (clé primaire, types, contrainte `NOT NULL`), on la crée d'abord avec `CREATE TABLE`, puis on importe avec `if_exists='append'` :

```
import sqlite3
import pandas as pd

DB = '/content/drive/MyDrive/TP_SQL_Dataset/tp_sql.db'   # base stockée sur Drive
conn = sqlite3.connect(DB)

conn.execute('DROP TABLE IF EXISTS donnees')
conn.execute('''
CREATE TABLE donnees (
    date_obs         TEXT PRIMARY KEY,
    annee            INTEGER NOT NULL,
    decennie         TEXT    NOT NULL,
    securities       INTEGER,
    nbr1_account     INTEGER,
    nbr2_account     INTEGER,
    cva_account      INTEGER,
    cva_nbr2_account INTEGER,
    total_holdings   INTEGER NOT NULL,
    signe_cva        TEXT
)''')

df.to_sql('donnees', conn, if_exists='append', index=False)
conn.commit()
```

4.3 Vérifications

```
pd.read_sql_query('SELECT COUNT(*) AS nb_lignes FROM donnees;', conn)
pd.read_sql_query('PRAGMA table_info(donnees);', conn)   # schéma
```

Nombre de lignes importées : 43 (= 43 lignes du DataFrame : l'import est complet). Schéma obtenu :

cid	name	type	notnull	pk
0	date_obs	TEXT	0	1
1	annee	INTEGER	1	0
2	decennie	TEXT	1	0
3	securities	INTEGER	0	0
4	nbr1_account	INTEGER	0	0
5	nbr2_account	INTEGER	0	0
6	cva_account	INTEGER	0	0
7	cva_nbr2_account	INTEGER	0	0
8	total_holdings	INTEGER	1	0
9	signe_cva	TEXT	0	0

```
SELECT date_obs, annee, decennie, nbr1_account, cva_account, total_holdings
FROM donnees
LIMIT 5;
```

date_obs	annee	decennie	nbr1_account	cva_account	total_holdings
1984-12-31	1984	1980s	1 718 233 650	412 091 784	2 130 328 115
1985-12-31	1985	1980s	1 697 761 841	390 544 083	2 088 308 602
1986-12-31	1986	1980s	2 686 016 541	0	2 686 028 968
1987-12-31	1987	1980s	2 686 016 541	0	2 686 028 970
1988-12-31	1988	1980s	2 534 826 491	151 190 050	2 686 028 971

Penser à fermer la connexion (`conn.close()`) en fin de notebook. La base étant écrite directement dans le dossier Drive, elle est conservée après la fin de la session Colab.

Partie 5 - Exploitation du dataset avec SQL (15 exercices)

Chaque requête est exécutée avec `pd.read_sql_query(requete, conn)`. Pour la lisibilité, les montants sont exprimés en **milliards de DTS (Md)** en divisant par 1e9 dans la requête.

Niveau 1 - Requêtes fondamentales

Exercice 1 - Afficher toutes les observations

Énoncé : Afficher l'ensemble de la table donnees.

```
SELECT *
FROM donnees;
```

Résultat obtenu (exécuté sur la base SQLite) :

date_obs	annee	decennie	securities	nbr1_account	nbr2_account	cva_account	cva_nbr2_account	total_holdings	signe_cva
1984-12-31	1984	1980s	0	1 718 233 650	2 170	412 091 784	511	2 130 328 115	Positif
1985-12-31	1985	1980s	0	1 697 761 841	2 178	390 544 083	500	2 088 308 602	Positif
1986-12-31	1986	1980s	0	2 686 016 541	12 427	0	0	2 686 028 968	Nul
1987-12-31	1987	1980s	0	2 686 016 541	12 429	0	0	2 686 028 970	Nul
1988-12-31	1988	1980s	0	2 534 826 491	11 730	151 190 050	700	2 686 028 971	Positif
1989-12-31	1989	1980s	0	2 869 514 461	13 279	-183 497 920	-849	2 686 028 971	Negatif
1990-12-31	1990	1990s	0	1 946 260 180	3 480	440 900 364	788	2 387 164 812	Positif
1991-12-31	1991	1990s	0	2 088 298 045	4 248	0	0	2 088 302 293	Nul
1992-12-31	1992	1990s	0	2 719 715 837	4 076	114 307 208	171	2 834 027 292	Positif
1993-12-31	1993	1990s	0	2 898 134 250	3 093	-25 701 622	-39	2 872 435 682	Negatif

(43 lignes au total, 10 affichées)

> **Lecture :** la table contient 43 lignes et 10 colonnes ; la série débute au 31/12/1984 et s'achève au 30/09/2026 (10 premières lignes affichées).

Exercice 2 - Sélectionner certaines colonnes

Énoncé : Afficher uniquement la date, le compte n° 1, le compte CVA et le total.

```
SELECT date_obs, nbr1_account, cva_account, total_holdings
FROM donnees;
```

Résultat obtenu (exécuté sur la base SQLite) :

date_obs	nbr1_account	cva_account	total_holdings
1984-12-31	1 718 233 650	412 091 784	2 130 328 115
1985-12-31	1 697 761 841	390 544 083	2 088 308 602
1986-12-31	2 686 016 541	0	2 686 028 968
1987-12-31	2 686 016 541	0	2 686 028 970
1988-12-31	2 534 826 491	151 190 050	2 686 028 971
1989-12-31	2 869 514 461	-183 497 920	2 686 028 971
1990-12-31	1 946 260 180	440 900 364	2 387 164 812
1991-12-31	2 088 298 045	0	2 088 302 293
1992-12-31	2 719 715 837	114 307 208	2 834 027 292
1993-12-31	2 898 134 250	-25 701 622	2 872 435 682

(43 lignes au total, 10 affichées)

> **Lecture :** on ne conserve que 4 colonnes sur 10 ; le nombre de lignes reste 43 (10 affichées). On voit déjà que le total est très proche du compte n° 1 : les autres composantes pèsent peu.

Exercice 3 - Limiter le nombre de résultats

Énoncé : Afficher les 5 premières lignes (date et total).

```
SELECT date_obs, total_holdings
FROM donnees
LIMIT 5;
```

Résultat obtenu (exécuté sur la base SQLite) :

date_obs	total_holdings
1984-12-31	2 130 328 115
1985-12-31	2 088 308 602
1986-12-31	2 686 028 968
1987-12-31	2 686 028 970
1988-12-31	2 686 028 971

Lecture : `LIMIT 5` renvoie les 5 premières lignes selon l'ordre de stockage. Comme `date_obs` est la clé primaire, on obtient ici les années les plus anciennes, mais sans `ORDER BY` cet ordre n'est pas garanti par le SQL.

Exercice 4 - Trier les observations

Énoncé : Afficher les 5 dates où le total des avoirs est le plus élevé.

```
SELECT date_obs, total_holdings
FROM donnees
ORDER BY total_holdings DESC
LIMIT 5;
```

Résultat obtenu (exécuté sur la base SQLite) :

date_obs	total_holdings
2017-12-31	26 658 649 795
2016-12-31	25 761 285 483
2018-12-31	25 664 311 571
2019-12-31	25 355 041 070
2020-12-31	23 687 993 514

Lecture : le record est de 26.66 Md de DTS fin 2017 ; les cinq plus hautes valeurs s'étalent de 2016 à 2020 : le sommet de la série est atteint juste après le saut de 2016.

Exercice 5 - Filtrer avec WHERE

Énoncé : Afficher les observations à partir de 2020.

```
SELECT date_obs, nbr1_account, total_holdings
FROM donnees
WHERE annee >= 2020
ORDER BY date_obs;
```

Résultat obtenu (exécuté sur la base SQLite) :

date_obs	nbr1_account	total_holdings
2020-12-31	24 255 978 451	23 687 993 514
2021-12-31	23 219 946 714	23 219 993 514
2022-12-31	22 490 606 714	22 490 653 514
2023-12-31	23 266 670 657	23 266 717 457
2024-12-31	23 000 140 155	23 000 186 955
2025-12-31	22 492 513 935	22 319 995 286
2026-09-30	22 629 291 897	22 453 992 205

Lecture : 7 lignes (2020 à 2025 et le 30/09/2026) ; le total se situe entre 22,3 et 23,7 Md de DTS, soit un niveau stable sur la période récente.

Niveau 2 - Agrégations

Exercice 6 - Compter les enregistrements

Énoncé : Calculer le nombre total d'enregistrements.

```
SELECT COUNT(*) AS nb_observations
FROM donnees;
```

Résultat obtenu (exécuté sur la base SQLite) :

nb_observations
43

Lecture : 43 enregistrements, cohérent avec le contrôle fait en Python (`df.shape[0]`).

Exercice 7 - Calculer une moyenne

Énoncé : Calculer le total moyen des avoirs, en Md de DTS.

```
SELECT ROUND(AVG(total_holdings)/1e9, 2) AS moyenne_md_sdr
FROM donnees;
```

Résultat obtenu (exécuté sur la base SQLite) :

moyenne_md_sdr
9,40

Lecture : la moyenne sur l'ensemble de la période est de 9.40 Md de DTS. Attention : une moyenne sur 42 ans mélange des niveaux très différents (voir l'exercice 9).

Exercice 8 - Minimum et maximum

Énoncé : Déterminer la plus petite et la plus grande valeur du total.

```
SELECT ROUND(MIN(total_holdings)/1e9, 3) AS min_md_sdr,
       ROUND(MAX(total_holdings)/1e9, 3) AS max_md_sdr
FROM donnees;
```

Résultat obtenu (exécuté sur la base SQLite) :

min_md_sdr	max_md_sdr
0,862	26,659

Lecture : minimum = 0.862 Md de DTS (31/12/1998), maximum = 26.659 Md de DTS (31/12/2017). Le maximum vaut environ 31 fois le minimum.

Exercice 9 - Regrouper avec GROUP BY

Énoncé : Calculer le nombre d'observations et le total moyen par décennie.

```
SELECT decennie,
       COUNT(*) AS nb_obs,
       ROUND(AVG(total_holdings)/1e9, 2) AS moyenne_md_sdr
FROM donnees
GROUP BY decennie
ORDER BY decennie;
```

Résultat obtenu (exécuté sur la base SQLite) :

decennie	nb_obs	moyenne_md_sdr
1980s	6	2,49
1990s	10	2,36
2000s	10	5,28
2010s	10	15,22
2020s	7	22,92

Lecture : le total moyen est quasi stable dans les années 1980-1990 (2,49 puis 2,36 Md), puis il double à peu près entre 1990s et 2000s (5,28), triple en 2010s (15,22) et atteint 22,92 Md dans les années 2020.

Exercice 10 - Filtrer les groupes avec HAVING

Énoncé : Ne conserver que les décennies dont la moyenne du total dépasse 10 Md de DTS.

```
SELECT decennie,
       ROUND(AVG(total_holdings)/1e9, 2) AS moyenne_md_sdr
FROM donnees
GROUP BY decennie
HAVING AVG(total_holdings) > 10e9;
```

Résultat obtenu (exécuté sur la base SQLite) :

decennie	moyenne_md_sdr
2010s	15,22
2020s	22,92

Lecture : seules les décennies 2010s et 2020s passent le seuil. `WHERE` filtre les lignes avant le regroupement ; `HAVING` filtre les groupes après le calcul de l'agrégat (ici l'agrégat est `AVG(total_holdings)`, qui ne peut pas figurer dans un WHERE).

Niveau 3 - Analyses avancées

Exercice 11 - Combiner AND et OR

Énoncé : Depuis 2015, lister les observations pour lesquelles le compte CVA est négatif ou le total dépasse 25 Md de DTS.

```
SELECT date_obs, cva_account, total_holdings
FROM donnees
WHERE annee >= 2015
  AND (cva_account < 0 OR total_holdings > 25e9)
ORDER BY date_obs;
```

Résultat obtenu (exécuté sur la base SQLite) :

date_obs	cva_account	total_holdings
2016-12-31	442 784 532	25 761 285 483
2017-12-31	-314 016 146	26 658 649 795
2018-12-31	1 200 179 133	25 664 311 571
2019-12-31	842 662 408	25 355 041 070
2020-12-31	-568 031 737	23 687 993 514
2025-12-31	-172 565 449	22 319 995 286
2026-09-30	-175 346 492	22 453 992 205

Lecture : 7 lignes sur 2016-2020 et 2025-2026. Les parenthèses sont indispensables : sans elles, `AND` serait prioritaire sur `OR` et le résultat changerait. 2016, 2018 et 2019 apparaissent grâce au critère « total > 25 Md » ; 2017 satisfait les deux, 2020, 2025 et 2026 seulement le critère CVA négatif.

Exercice 12 - BETWEEN et IN

Énoncé : Lister les années entre 2005 et 2010 (inclus) qui sont aussi dans {2005, 2008, 2009, 2010}.

```
SELECT date_obs, ROUND(total_holdings/1e9, 2) AS total_md_sdr
FROM donnees
WHERE annee BETWEEN 2005 AND 2010
  AND annee IN (2005, 2008, 2009, 2010)
ORDER BY date_obs;
```

Résultat obtenu (exécuté sur la base SQLite) :

date_obs	total_md_sdr
2005-12-31	5,40
2008-12-31	6,77
2009-12-31	6,52
2010-12-31	6,41

Lecture : `BETWEEN` inclut ses deux bornes ; `IN` teste l'appartenance à une liste. Les années 2006 et 2007 sont écartées par le `IN`. On observe un pic de 6,77 Md en 2008 puis une légère baisse en 2009 et 2010.

Exercice 13 - Classement avec ORDER BY

Énoncé : Établir le top 10 des années selon le total des avoirs.

```
SELECT annee,
       ROUND(total_holdings/1e9, 2) AS total_md_sdr
FROM donnees
ORDER BY total_holdings DESC
LIMIT 10;
```

Résultat obtenu (exécuté sur la base SQLite) :

annee	total_md_sdr
2017	26,66
2016	25,76
2018	25,66
2019	25,36
2020	23,69
2023	23,27
2021	23,22
2024	23,00
2022	22,49
2026	22,45

Lecture : les 10 meilleures années sont toutes postérieures à 2015 : le classement est dominé par 2016-2020, suivi de 2021-2024 puis 2026. Ce résultat reflète le niveau structurellement plus élevé depuis 2016.

Exercice 14 - Sous-requête

Énoncé : Lister les années dont le total est supérieur à la moyenne générale de la période (calculée par une sous-requête).

```
SELECT annee, ROUND(total_holdings/1e9, 2) AS total_md_sdr
FROM donnees
WHERE total_holdings > (SELECT AVG(total_holdings) FROM donnees)
ORDER BY annee;
```

Résultat obtenu (exécuté sur la base SQLite) :

annee	total_md_sdr
2016	25,76
2017	26,66
2018	25,66
2019	25,36
2020	23,69
2021	23,22
2022	22,49
2023	23,27
2024	23,00
2025	22,32
2026	22,45

Lecture : 11 années (2016 à 2026), toutes consécutives : depuis 2016 le total est toujours supérieur à la moyenne de 9,40 Md. Avant 2016 aucune année ne dépasse cette moyenne (maximum 9,33 Md en 2014).

Exercice 15 - Analyse synthétique (question économique)

Question : comment la position du FMI en monnaie chinoise a-t-elle évolué décennie par décennie, et avec quelle fréquence l'ajustement CVA a-t-il été négatif ?

```
SELECT decennie,
       COUNT(*) AS nb_obs,
       ROUND(MIN(total_holdings)/1e9, 2) AS min_md,
       ROUND(AVG(total_holdings)/1e9, 2) AS moy_md,
       ROUND(MAX(total_holdings)/1e9, 2) AS max_md,
       SUM(CASE WHEN cva_account < 0 THEN 1 ELSE 0 END) AS nb_cva_negatif
FROM donnees
GROUP BY decennie
ORDER BY decennie;
```

Résultat obtenu :

decennie	nb_obs	min_md	moy_md	max_md	nb_cva_negatif
1980s	6	2,09	2,49	2,69	1
1990s	10	0,86	2,36	3,00	4
2000s	10	3,22	5,28	7,56	3
2010s	10	6,41	15,22	26,66	6
2020s	7	22,32	22,92	23,69	3

Lecture : le niveau moyen passe de 2,49 Md (années 1980) à 22,92 Md (années 2020), soit environ 9 fois plus. La dispersion (min-max) est très faible dans les années 1980 (2,09 à 2,69) et devient énorme dans les années 2010 (6,41 à 26,66) : c'est la signature d'une rupture dans la série en 2016 (voir partie 6). Le compte CVA est négatif sur 17 observations au total, dont 6 sur 10 dans les années 2010 : le signe de cet ajustement n'est pas lié au niveau du total.

Exercice facultatif - Variations d'une année à l'autre (fonction de fenêtre LAG)

Énoncé : identifier les 5 plus fortes variations annuelles du total (en valeur absolue).

```
SELECT annee, total_md, variation_md FROM (
  SELECT annee,
         ROUND(total_holdings/1e9, 2) AS total_md,
         ROUND((total_holdings - LAG(total_holdings) OVER (ORDER BY date_obs))/1e9, 2) AS variation_md
  FROM donnees)
WHERE variation_md IS NOT NULL
ORDER BY ABS(variation_md) DESC
LIMIT 5;
```

annee	total_md	variation_md
2016	25,76	16,50
1999	3,00	2,14
2006	7,37	1,98
2020	23,69	-1,67
2005	5,40	1,17

Lecture : la plus forte variation est +16,50 Md en 2016 (de 9,26 à 25,76 Md), soit plus de 7 fois la deuxième (+2,14 Md en 1999). Les autres variations restent inférieures à 2,2 Md en valeur absolue. Le saut de 2016 est donc **un point singulier**, et non une évolution progressive. Remarque : la dernière observation étant au 30/09/2026, la variation 2025 → 2026 porte sur 9 mois seulement.

Partie 6 - Analyse et interprétation

6.1 Cinq requêtes significatives, résultats et interprétation

N°	Requête retenue	Pourquoi elle est significative
1	Ex. 9 / 15 : agrégats par décennie	Résume 42 ans de données en 5 lignes et montre la tendance de fond.
2	Ex. 8 : minimum et maximum	Donne l'amplitude de la série (de 0,86 à 26,66 Md).
3	Ex. 14 : sous-requête (total > moyenne)	Révèle que toutes les années au-dessus de la moyenne sont postérieures à 2015.
4	Facultatif : variations avec LAG	Isole la rupture de 2016 (+16,50 Md).
5	Ex. 11 : conditions AND/OR sur le CVA	Montre que le CVA change de signe d'une année à l'autre, indépendamment du niveau.

Tableau Pandas n° 1 - Moyenne, minimum et maximum du total par décennie (en Md de DTS)

decennie	nb_obs	min_md	moy_md	max_md
1980s	6	2,09	2,49	2,69
1990s	10	0,86	2,36	3,00
2000s	10	3,22	5,28	7,56
2010s	10	6,41	15,22	26,66
2020s	7	22,32	22,92	23,69

Interprétation. Le total moyen croît de décennie en décennie, mais la progression n'est pas régulière : une quasi-stabilité autour de 2,4 Md pendant 1984-1999, une croissance modérée dans les années 2000 (5,28 Md) puis une forte hausse dans les années 2010 (15,22 Md) et 2020 (22,92 Md). Sur la hausse totale d'environ 20,3 Md entre 1984 et septembre 2026, 16,5 Md surviennent donc en une seule année, 2016.

Tableau Pandas n° 2 - Amplitude de la série

min_md_sdr	max_md_sdr
0,862	26,659

Interprétation. Le minimum (0,862 Md) est observé en 1998 et le maximum (26,659 Md) en 2017. Le rapport d'environ 31 entre les deux extrêmes montre qu'une moyenne globale (9,40 Md) est peu représentative : elle ne correspond à aucune période réelle de la série.

Tableau Pandas n° 3 - Années au-dessus de la moyenne (sous-requête)

annee	total_md_sdr
2016	25,76
2017	26,66
2018	25,66
2019	25,36
2020	23,69
2021	23,22
2022	22,49
2023	23,27
2024	23,00
2025	22,32
2026	22,45

Interprétation. Les 11 années au-dessus de la moyenne forment un bloc continu 2016-2026. Sur ce bloc, le total culmine en 2017 (26,66 Md) puis décline de façon irrégulière jusqu'à 22,45 Md au 30/09/2026, soit environ -4,2 Md par rapport au sommet.

Tableau Pandas n° 4 - Les 5 plus fortes variations annuelles

annee	total_md	variation_md
2016	25,76	16,50
1999	3,00	2,14
2006	7,37	1,98

annee	total_md	variation_md
2020	23,69	-1,67
2005	5,40	1,17

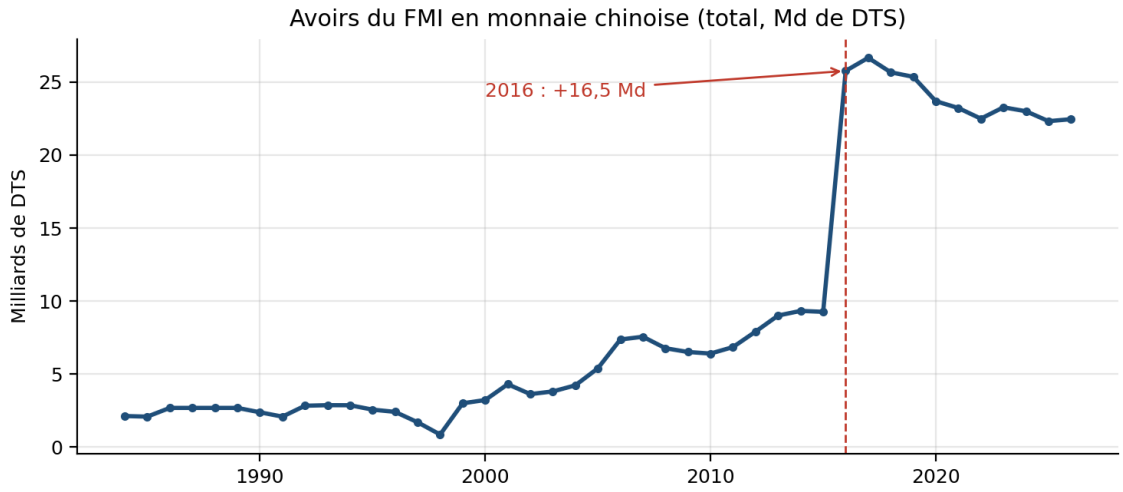
Interprétation. La variation de 2016 (+16,50 Md) est sans commune mesure avec les autres. Une hypothèse économique plausible, à vérifier dans la documentation du FMI et non établie par les seules données, est un changement institutionnel (par exemple une révision des quotes-parts du FMI entrée en vigueur à cette période) plutôt qu'un phénomène de marché : un saut unique de cette taille, suivi d'un plateau, ressemble à un changement de niveau.

Tableau Pandas n° 5 - Observations depuis 2015 avec CVA négatif ou total > 25 Md

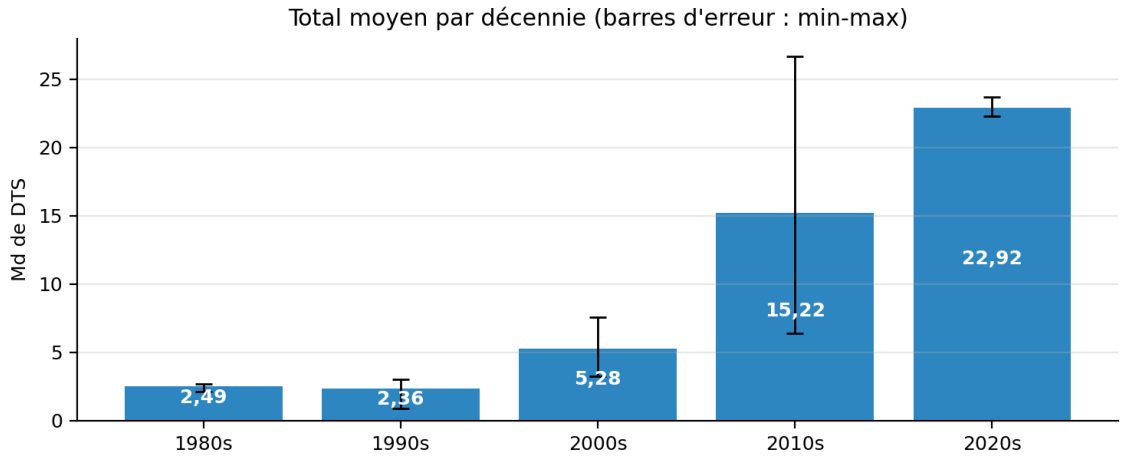
date_obs	cva_account	total_holdings
2016-12-31	442 784 532	25 761 285 483
2017-12-31	-314 016 146	26 658 649 795
2018-12-31	1 200 179 133	25 664 311 571
2019-12-31	842 662 408	25 355 041 070
2020-12-31	-568 031 737	23 687 993 514
2025-12-31	-172 565 449	22 319 995 286
2026-09-30	-175 346 492	22 453 992 205

Interprétation. Le CVA alterne entre positif et négatif d'une année sur l'autre (par exemple +0,44 Md en 2016, -0,31 en 2017, +1,20 en 2018, +0,84 en 2019, -0,57 en 2020). Son ordre de grandeur (au plus 1,2 Md) reste faible face au total (plus de 22 Md), mais il explique à lui seul les écarts entre le compte n° 1 et le total.

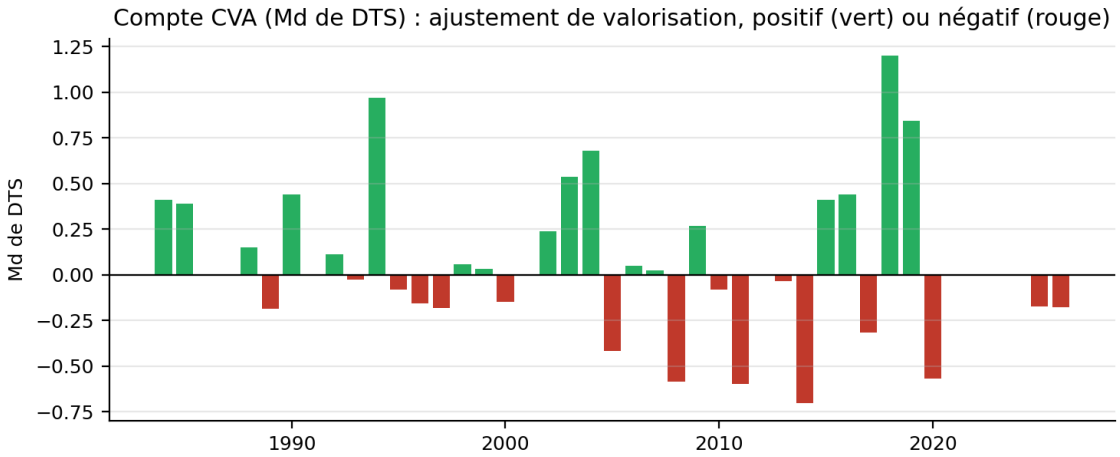
6.2 Graphiques Matplotlib



***Graphique 1** - Évolution du total des avoirs : stabilité jusqu'en 1999, croissance irrégulière jusqu'en 2015, saut en 2016, puis légère érosion.*



***Graphique 2** - Total moyen par décennie, avec l'étendue min-max : la barre d'erreur des années 2010 est la plus grande, signe de la rupture de 2016.*



Graphique 3 *- Compte CVA par année : alternance de signes ; les ajustements positifs les plus élevés ont lieu en 1994 (0,97 Md) et 2018 (1,20 Md).*

```
import matplotlib.pyplot as plt
d = pd.read_sql_query(
    'SELECT annee, total_holdings/1e9 AS total_md FROM donnees ORDER BY date_obs', conn)
plt.figure(figsize=(8, 3.5))
plt.plot(d.annee, d.total_md, marker='o')
plt.axvline(2016, color='red', linestyle='--')
plt.title('Avoirs du FMI en monnaie chinoise (Md de DTS)')
plt.ylabel('Milliards de DTS'); plt.grid(alpha=.3); plt.show()
```

6.3 Trois conclusions

1. Une croissance par paliers, et non régulière. Le total passe d'environ 2,1 Md en 1984 à 22,45 Md en septembre 2026 (x10,5), mais l'essentiel de la hausse se concentre en 2016 (+16,50 Md en un an) ; avant, la série varie de moins de 2,2 Md par an.

2. Une moyenne globale trompeuse. Moyenne de 9,40 Md sur 42 ans, alors que 11 années sur 43 seulement la dépassent et qu'elles sont toutes postérieures à 2015. Une analyse par décennie ou par sous-période est indispensable.

3. Une composition dominée par un seul compte. Le compte n° 1 représente l'essentiel du total ; le compte n° 2 est négligeable (au plus 50 582 DTS) et `securities` est toujours nul. L'ajustement CVA est faible (au plus 1,2 Md en valeur absolue) mais change de signe régulièrement (17 observations négatives sur 43).

Limite des conclusions : avec 43 points annuels et une seule série, l'analyse est descriptive. Elle ne permet pas d'expliquer les causes des variations ; les hypothèses économiques proposées doivent être confrontées à la documentation du FMI.

Partie 7 - Livrables et synthèse

7.1 Liste des livrables

N°	Livrable	Statut
1	Notebook Google Colab (.ipynb)	À assembler à partir des cellules de code des parties 2 à 6 de ce document (non fourni séparément).
2	Dataset original	`extportal.xls` (fichier source FMI, non modifié).
3	Base SQLite (.db)	`tp_sql.db`, fournie avec ce PDF (table `donnees`, 43 lignes).
4	15 requêtes SQL et résultats	Partie 5 (+ 1 exercice facultatif).
5	Synthèse d'une page	Paragraphe ci-dessous.

7.2 Synthèse d'une page

Objectif. Parcourir toute la chaîne source → Drive → Colab → Pandas → SQLite → SQL → analyse, à partir d'un fichier du FMI sur les avoirs du FMI en monnaie chinoise (43 observations, 1984-2026).

Principaux résultats. Le total des avoirs a une moyenne de 9,40 Md de DTS, avec un minimum de 0,862 Md (1998) et un maximum de 26,659 Md (2017). Par décennie, la moyenne passe de 2,49 Md (années 1980) à 22,92 Md (années 2020). Une rupture nette apparaît en 2016 : +16,50 Md en un an, soit plus de 7 fois la deuxième plus forte variation. Depuis 2016, toutes les années dépassent la moyenne générale, avec un léger recul de 26,66 Md (2017) à 22,45 Md (30/09/2026). Le compte n° 1 domine ; le CVA est de faible amplitude mais de signe variable.

Difficultés rencontrées et solutions.

Difficulté	Solution
Le fichier .xls n'est pas un classeur Excel (texte à tabulations) : `read_excel` échoue.	`pd.read_csv(sep='\t', skiprows=4, thousands=',')`.
Nombres écrits avec des virgules (22,629,291,897) lus comme du texte.	Option `thousands=','`.
Dates en anglais (« December 31, 2024 »).	`pd.to_datetime(format='%B %d, %Y')`, puis texte ISO pour SQLite.
Moins de 100 lignes et aucune variable catégorielle.	Écart justifié (partie 1) ; création de `decennie` et `signe_cva`.
Montants à 11 chiffres illisibles.	Division par 1e9 dans les requêtes (milliards de DTS).
Interprétation économique de la rupture de 2016.	Hypothèse formulée mais clairement signalée comme à vérifier.

7.3 Erreurs fréquentes à éviter

- Oublier `drive.mount` ou se tromper de chemin (`MyDrive` avec majuscules).
- Lancer `to_sql` avec `if_exists='append'` deux fois : doublon de clé primaire (`IntegrityError`) ; relancer `DROP TABLE` avant.
- Mettre une agrégation dans `WHERE` au lieu de `HAVING`.
- Oublier les parenthèses autour d'un `OR` combiné à un `AND`.
- Utiliser `LIMIT` sans `ORDER BY` et croire que l'ordre est garanti.

7.4 Rappel de la grille d'évaluation (sur 20)

Critère	Points	Où le trouver dans ce document
Choix et justification du dataset	2	Partie 1
Téléchargement et stockage sur Drive	3	Partie 2
Exploration et préparation des données	3	Partie 3
Création de la base SQLite	3	Partie 4
Qualité des requêtes SQL	5	Partie 5
Analyse et interprétation	3	Partie 6
Organisation et documentation du notebook	1	Structure d'ensemble
Total	**20**	